

Linux Shell Scripting

System Health Check & Backup Scripts



Outlines:

- Introduction
- Purpose and Functionality of Each Script
- Scripts Explanation
- Performance Analysis

Note: We will use “WSL” to run Linux on Windows as a Virtual Machine.

Introduction

This report will discuss two scripts:

- **Backup Script**
- **System Health Check Script**

In each script, we will automate tasks that usually performed manually, so for example we usually copy the directory that we want to back it up and paste this directory to another location where we want to be in, but with the **Backup** Script, we will see how we can automate this process with more efficiency and with better performance.

And with the **System Health Check** Script, we will see how we will automate the process of checking the health of the system, with some recommendations to be made.

Output formatting:

In both scripts, we may see some **colors** that help to understand the output of each script (especially in the Health Script), and for these colors, I've used the **ANSI** escape codes, which are sequences of characters used to control text formatting, and the following are the colors that I've used in my both scripts.

We can use these colors when printing the output (using 'echo' or 'printf') by adding one or more colors before the content we want to print.

```
#colors
red='\033[31m'
green='\033[32m'
yellow='\033[33m'
blue='\033[34m'
orange='\033[38;2;255;120;0m'
bold='\033[1m'
reset='\033[0m'
```

Here **"reset"** indicates the default text formatting of the console, and we can use it after the content we want to print when we are done with this color.

Now we will discuss the purpose and functionality of each script.

Purpose and Functionality of Each Script

- **Backup Script Functionality**

In this script, we will automate the process of making a backup of a user specified directory with some preferences like:

- If the user wants to **specify the location** of the backup or to keep the default location (at '/usr/root').
- If the user wants to **compress the backup or keep it** as a directory (not compressed), and if the user chooses to compress the backup, we will Zip file format.

We will also check the status of each backup that has been created using the **log.txt** file, because this backup process may be done successfully or to be failed due to some error, so the log.txt file will save the status and other information (we will see more details later) about the backup process.

- **System Health Check Functionality**

In this script, we will automate a couple of system checks including:

- **Disk Space** Check

- Checks the available disk space and identifies any disks that have less than 20% free space.

- **Memory Usage** Check

- Check the system's memory usage and alerts if the usage exceeds 80%.

- **Running Services** Check

- Lists all currently running services on the system.

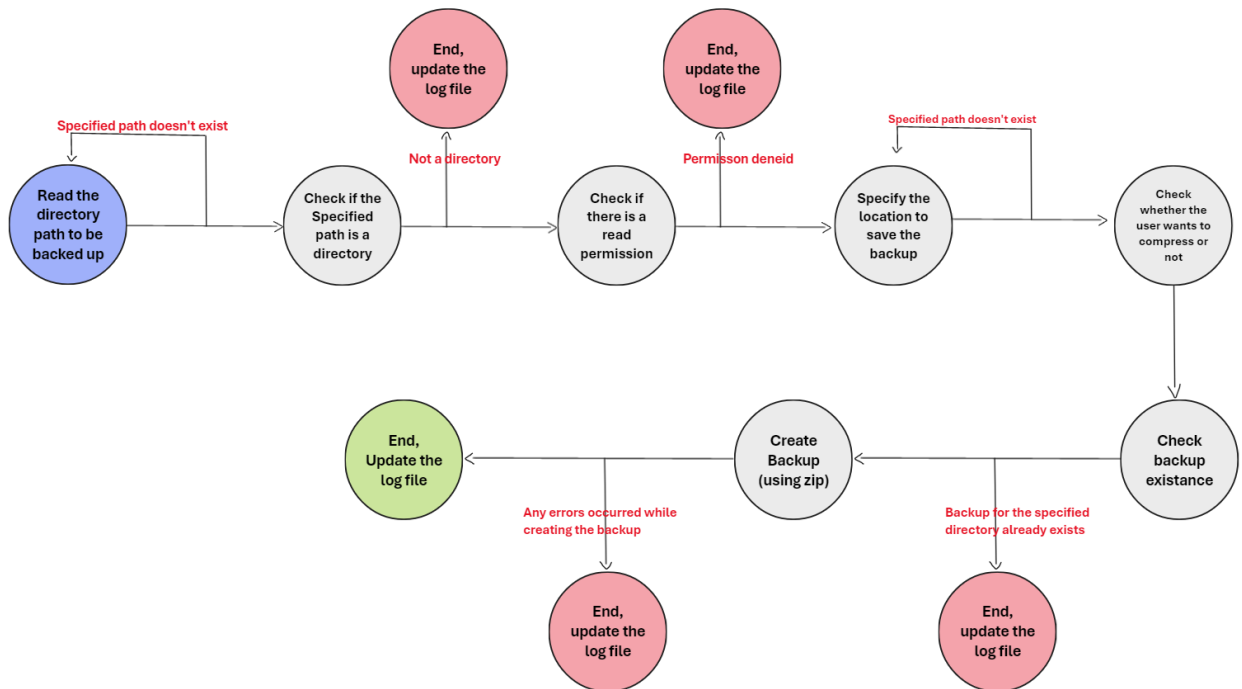
- **Recent System** Updates Check

- Identify recent updates and installations.

Scripts Explanation

- **Backup Script Explanation:**

First, we will see the flow of the process, and then we will explain each function in the script.



As we can see, the process starts by **reading the path** to the directory, check if the specified **path is a directory**, then check if we **can read this directory**, specify the **location** where we want to **save the backup**, then check whether the user wants to **compress the backup or not**, check the **existence of the backup** in the specified location of the backup, **create the backup**, and then **end** the process.

Next, we will discuss each step in detail and see how we can handle any errors that could happen during the process of creating the backup.

- **Reading the directory path to be backed up:**

We will start the execution by reading the path to the directory (as shown in the **blue** circle) that the user wants to create a backup of it, then check if the specified path is a valid path or not, if not, we will keep asking the user to enter a valid path, until he enters a valid path, as shown below:

```
read -p "Directory path to be backed up: " DIR
while [ ! -d ${DIR} ] && [ ! -f ${DIR} ]; do
    read -p "Please enter a valid path: " DIR
done
```

The condition in the while loop is false if the specified path is not a directory ('! -d') and not a file ('! -f').

- **Check if the specified path is a directory:**

If the user enters a valid path, then we will check if this path is a directory, if not we will exit the script and save the error that we faced (here the error is that the **specified path is not a directory**) and then update the **log** file with this error (we will discuss the log file later).

```
if [ -d "$DIR" ]; then
```

If yes, we will continue, otherwise:

```
else
    error="Specified path is not a directory"
fi
```

'-d' here inside the 'if' statement means "**is it directory?**", which returns true if the given path is a directory, false otherwise.

- **Check if there is a read permission:**

If the specified path is a directory, then we will check if there is a read permission, if the **permission denied**, then we will exit the script, save the error that we have faced and then update the log file.

```
check_permission () {  
    path=$1  
    find "${path}" ! -readable -printf "No read permission for : %p\n"  
}
```

The above function takes the path to the directory and checks if there is a read permission to this directory and all its subdirectories and files or not using the '**find**' command.

If the '**find**' command returns nothing, then there is read permission to this directory, and we will continue to next step, otherwise, we will exit, save the error and update the log file.

- **Specify the location to save the backup:**

Which is specifying the location that the user wants to save the backup in (at some directory).

```
read -p "Directory path where you want to save the backup ['d' for default]: " directory_path  
while [[ ${directory_path,,} != "d" ]] && [ ! -d ${directory_path} ]; do  
    read -p "Please enter a valid directory path ['d' for default path]: " directory_path  
done
```

'\${**directory_path**,,}' here converts the "**directory_path**" to lower case, so that whether the user enters "**d**" or "**D**" it is the same, and here 'd' as we can see indicates that the user chooses the default location to save the backup (which is under the /usr/root/ path), but if the user enters a path, the first condition be **true**, but the second condition will not become true, until the user enters a valid directory path

Note: We will prevent creating a backup of directory "A" within directory "A" itself or its subdirectories.

If the user enters a valid directory path, then we will continue to the next step.

- **Check whether the user wants to compress or not:**

Next, we will ask the user whether he wants no compress the backup or not using the following function:

```
check_compression () {  
    read -p "Do you want to compress the backup into a zip file [Y/N]? " check_comp  
    Check_compL="${check_comp,,}"  
    [[ $Check_compL == "yes" || $Check_compL == "y" ]] && comp=1  
}
```

In the second line inside the function, we will convert the '**check_comp**' variable (which is the string that the user enters and can be "yes/y" or "no/n" in any case) into small case using '\${**check_comp**,,}', and with this, even if the user enters a capital letter, this command will handle it so that we can compare inside the 'short' if statement.

The third line inside the function starts with a condition (which we may name it as a 'short if', and we have used **[[...]]** format because we are comparing strings), this condition checks if the user enters "yes or y", if true, then we will change the global variable '**comp**' to be '1' (the value of '**comp**' is 0 by default and this variable will be used later on to decide whether to compress the backup or not).

- **Check backup existence:**

Next, we will check the existence of the specified directory to be backed up, if the same type exists in the desired location (the user wants to compress the backup and the compressed backup exists in the desired location and vice versa), then we will exit the script, save the error that we faced and update the log file.

Next, we will see two variables that will help us to check, the first one is "**root**" variable, which is the **location** where the user wants to save the backup, and the second one is "**str_path**", which is the **concatenation** of all the super directories of the desired directory to be backed up, for example, if the path to the directory is **"/home/ahmad/Desktop/Test"** then '**str_path**' string will be:

str_path="homeahmadDesktopTest", and so on.

And this can be done using the following function:

```
path_to_str() {  
    IFS='/'  
    path=$1  
    str_path=""  
    read -ra directories <<< $path  
    for dir in ${directories[@]}; do  
        if [ -n dir ]; then  
            str_path="$str_path$dir"  
        fi  
    done  
    echo "${str_path}"  
}
```

Which is saving the super directories into an array separated by '/' and then iterate over this array and concatenate each super directory with the '**str_path**' string.

So, we will check if the compressed backup does not exist using:

```
if [ ! -f "${root}/compressed/${str_path}_Backup.zip" ]; then  
    backup_path="${root}/compressed/${str_path}_Backup.zip"  
    zip -r $backup_path $DIR  
else  
    error="Compressed Backup for the directory ${DIR} is already exists"  
fi
```

Or if the directory backup (not compressed) does not exist using:

```
if [ ! -d "${root}/${str_path}_Backup" ]; then  
    backup_path="${root}/${str_path}_Backup"  
    mkdir $backup_path  
    cp -r ${DIR} $backup_path  
else  
    error="Backup for the directory ${DIR} is already exists"  
fi
```

Then we will continue to make the backup.

- **Create Backup:**

Finally, we will create the backup, and there are two cases:

- **Create a compressed backup:**

If the user wants to create a compressed backup, then we will first check if there is a folder named “**compressed**” in the desired location, if not, we will create one, as the follow:

```
if [ ! -d "${root}/compressed" ]; then  
    mkdir "${root}/compressed"  
fi
```

Then we will create the compressed backup, which will be done by compressing the directory and all its subdirectories and files using the **zip** algorithm.

```
backup_path="${root}/compressed/${str_path}_Backup.zip"  
zip -r $backup_path $DIR
```

‘-r’ here means to recursively include all the subdirectories and files that are contained by this directory.

- **Create a directory backup (not compressed):**

If the user wants to create a backup that is not compressed, then we will directly create the backup using ‘**cp**’ command, which is copying the backup itself to another location, as follow:

```
backup_path="${root}/${str_path}_Backup"  
mkdir $backup_path  
cp -r ${DIR} $backup_path
```

So, we will copy the backup into the ‘**backup_path**’, and with this we obtained a backup of the desired directory in the desired location.

In both cases, if the backup of the same type (compressed or not compressed) exists, then we will tell the user to replace it or not, and if there are no errors while creating the backup, we will just continue.

- **End, update the log file:**

At the end of this process, there are just two things left, first we will print the error (if we've faced any errors during the backup process), then we will update the log file.

Updating the log file is two cases:

- If the backup process successfully done without facing any errors we will report about the following:
 - The backup has been compressed or not
 - The size of the backup
 - The creation date of the backup
 - The location where the backup has been saved

```
if [[ $error == "" ]]; then
    echo "    status      : Success" >> $logFile
    echo -n "    compressed : " >> $logFile
    [ $comp -eq 1 ] && echo "Yes" >> $logFile || echo "No" >> $logFile
    echo "    size       : $(du -sh $backup_path | awk '{print $1}')" >> $logFile
    echo "    date        : $(date)" >> $logFile
    echo "    location    : ${root}" >> $logFile
    echo -e "\n$(colored_echo "Everything is done" "${green}${bold}"), check the ${root} directory\n"
```

'logFile' is the path to the log file.

- Or if the backup hasn't been successfully done and we face some errors during the backup process we will report about:
 - The error that we have faced during the backup process
 - The date when we tried to make the backup

```
echo "    status      : Failed" >> $logFile
echo "    error       : ${error}" >> $logFile
echo "    date        : $(date)" >> $logFile
echo -e "\n$(colored_echo "Something went wrong while creating a backup" "${red}${bold}") for the directory '${DIR}'."
echo "Please check the '${logFile}' file for detailed error information\n"
```

And that's all about the backup script, next we will see an explanation about the System Health Check.

- **System Health Check Script Explanation:**

First, let us talk about the functions that will help us to create the **health** script (helper functions):

- **Colored echo:**

This function is a simple function that prints the output using the command '**echo**' but with specified color as the following:

```
colored_echo(){  
    text=$1  
    color=$2  
    echo -e "${color}${text}${reset}"  
    sleep 0.001  
}
```

As we can see, this function takes **text** and **color**, and this color could be one of the following colors:

```
#colors  
red='\033[31m'  
green='\033[32m'  
yellow='\033[33m'  
blue='\033[34m'  
orange='\033[38;2;255;120;0m'  
bold='\033[1m'  
reset='\033[0m'
```

These colors are global variables that use the **ANSI** code to modify the color of the text being printed in the console, and this can be done by printing the color before the text that we want to color it (as shown above), and then we can use the color known as '**reset**' which will reset the format of the output to its default formatting.

- **Separator:**

We will use this function to print a line of #’s before entering any of the four checks, as the following:

```
#####
```

And this can be done by the following function:

```
separator(){
    echo -en "${bold}"
    for i in {1..76}; do
        echo -n "#"
        sleep 0.005
    done
    echo -e "${reset}"
}
```

Here we use the color '**bold**' which prints the line bolded.

- **Capitalize:**

This function simply takes a word, capitalize the first character of it and converts rest to lowercase:

```
capitalize() {
    echo -n "$1" | awk '{print toupper(substr($0,1,1)) tolower(substr($0,2))}'
}
```

- **Formatted line:**

We will use this function to print in both '**disk space**' and '**memory usage**', which will print a given line with a given color in such way that each two words will be separated by '\t', and with a sleep of 0.01 seconds after each line.

Also, this function will check if there is a disk running in low space when we check the '**disk space**'.

Next, let us see the main functions:

- **Printing the Header:**

In the beginning of the script, we will print a header as the following:

```
===== [ System Health Check ] =====  
  
This report will include checks for:  
1. Disk Space  
2. Memory Usage  
3. Running Services  
4. Recent System Updates  
  
===== [ Starting Checks... ] =====
```

Which tells the checks that will be done in this script, and we will use the **'helper functions'** to print this header as the following:

```
#Header  
print_header(){  
    echo -e "\n===== [ $(colored_echo "System Health Check" "${green}${bold}") ] =====\n"  
    colored_echo "This report will include checks for:" "${bold}"  
    echo "1. $(colored_echo "Disk Space" "${yellow})"  
    echo "2. $(colored_echo "Memory Usage" "${yellow})"  
    echo "3. $(colored_echo "Running Services" "${yellow})"  
    echo "4. $(colored_echo "Recent System Updates\n" "${yellow})"  
    echo -e "===== [ $(colored_echo "Starting Checks..." "${green}${bold}") ] =====\n\n"}
```

We can see that we've utilized the **'colored_echo'** helper function to color the output of the header as shown above.

- **Check Disk Space:**

This function will check the available disk space and identify any disks that have less than **20%** free space (using the **'formatted line'** helper function), and this can be done using the **'df'** command (disk filesystem), which displays the information about the disk space on the filesystem (available/used disk space) by provides details like the type of each filesystem, total size, used size, available size, used percentage regarding to the total size, location of the filesystem (mount), we can use **'-h'** for human readability, **'-T'** to include the type of the filesystem.

When executing the command, this will be displayed the following:

```
ahmdjaber@DESKTOP-MNB04TB:~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
none            1978816          4    1978812    1% /mnt/wsl
drivers        249419832 206639580  42780252  83% /usr/lib/wsl/drivers
none            1978816          0    1978816   0% /usr/lib/modules
none            1978816          0    1978816   0% /usr/lib/modules/5.15.153.1-microsoft-standard-WSL2
/dev/sdc        1055762868 2880444 999178952   1% /
none            1978816          0    1978816   0% /mnt/wslg
none            1978816          0    1978816   0% /usr/lib/wsl/lib
rootfs          1975464        2080    1973384   1% /init
none            1978816          0    1978816   1% /run
none            1978816          0    1978816   0% /run/lock
none            1978816          0    1978816   0% /run/shm
tmpfs           4096           0         4096   0% /sys/fs/cgroup
none            1978816          0    1978740   1% /mnt/wslg/versions.txt
none            1978816          0    1978740   1% /mnt/wslg/doc
C:\             249419832 206639580  42780252  83% /mnt/c
snapfuse         128           128          0 100% /snap/bare/5
snapfuse        76032         76032          0 100% /snap/core22/1380
snapfuse        93952         93952          0 100% /snap/gtk-common-themes/1535
```

And this is what we want when checking the disk space, but we will filter and format this output to print just the main disks like the following:

Filesystem	Type	Size	Used	Avail	Use%	Mounted_on
/dev/sdc	ext4	1007G	2.8G	953G	1%	/
none	tmpfs	1.9G	4.0K	1.9G	1%	/mnt/wsl
drivers	9p	238G	197G	42G	83%	/usr/lib/wsl/drivers
none	tmpfs	1.9G	0	1.9G	0%	/usr/lib/modules
none	overlay	1.9G	0	1.9G	0%	/usr/lib/modules/5.15.153.1-microsoft-standard-WSL2
none	tmpfs	1.9G	108K	1.9G	1%	/mnt/wslg
none	overlay	1.9G	0	1.9G	0%	/usr/lib/wsl/lib
rootfs	rootfs	1.9G	2.1M	1.9G	1%	/init
none	tmpfs	1.9G	816K	1.9G	1%	/run
none	tmpfs	1.9G	0	1.9G	0%	/run/lock
none	tmpfs	1.9G	0	1.9G	0%	/run/shm
tmpfs	tmpfs	4.0M	0	4.0M	0%	/sys/fs/cgroup
none	overlay	1.9G	76K	1.9G	1%	/mnt/wslg/versions.txt
none	overlay	1.9G	76K	1.9G	1%	/mnt/wslg/doc
C:\	9p	238G	197G	42G	83%	/mnt/c

And this can be done using the following function:

```
echo -e "${formatted_line "Filesystem Type Size Used Avail Use% Mounted_on" "${bold}"}"
echo -n "
formatted_line "${df -hT / | grep "dev")" "${orange}"
mapfile -t diskCheck <<< "${df -hT | grep -vE "snapfuse|tmpfs|/dev/sd")"
for line in "${diskCheck[@]}"; do
    if [[ ${line} == *"Size"* ]]; then
        continue
    fi
    formatted_line "${line}" "${reset}"
done
```

So, we will print the **‘/dev/.’** filesystems (which are the main filesystems) in the orange color.

And we will use the default format in the other filesystems.

Here **'mapfile'** is a command that converts a string of lines separated by **'\n'** into an indexed array, **'-t'** removes the trailing **'\n'** from each line in the string.

The command **'grep'** is a powerful command that we can use to filter the disks that we want to include (**'-v'** for filtering invert the match, **'-E'** to include multiple patterns inside the **'grep'** command).

So, we will iterate over the array that we create using the **'mapfile'** and print each line with a better format using the **'formatted_line'** helper function.

Next, we will check if there are any disks running with low available space, and this can be also done with the help of the **'formatted_line'** function as follows:

```
echo -e "\n"
if [ ${#low_disks} -ge 1 ]; then
    echo -e "\t|$(colored_echo "Disks with Low Available Space:" "${red}${bold}")"
    for disk in ${low_disks}; do
        echo -e "\t|\t- $(colored_echo "${disk}")"
    done
    echo -e "\n"
    echo -e "\t|$(colored_echo "Recommendations:" "${bold}${green}")"
    echo -e "\t|\t$(colored_echo "- Remove unnecessary files.")"
    echo -e "\t|\t$(colored_echo "- Uninstall apps that you don't use frequently.")"
else
    echo -e "\n\t$(colored_echo "|Everything goes well with the disks" "${bold}${green}")"
fi
echo -e "\n$(colored_echo "Done Checking Disk Space." "${blue}${bold})\n\n\n"
```

If we found any disks that running in low space, then we will alert the user about these disks as follows:

```
| Disks with Low Available Space:
|   - drivers
|   - C:
```

We will also alert the user about some actions and recommendations that may help him if there are any disks that running in low space as follows:

```
| Recommendations:
|   - Remove unnecessary files using the 'rm <file_name>' command.
|   - Uninstall packages or apps that you don't use frequently.
|   - You can use the command 'sudo apt-get remove <package_name>' to Uninstall apps.
```

If there are no disks that run in low space, we will just tell the user that everything is good.

- **Check Memory Usage:**

Which is checking the system's memory usage and alerts if the usage exceeds 80%, and this can be done using the command '**free**' which displays information about the memory usage including total amount of memory, used, free, shared, buff/cache and the available, we can use '**-m**' to display the data in megabytes, so we will check the memory usage using the following function:

```
check_memory_usage(){
    head="\t\t"
    head="${head}"
    head_mem=$(free | grep "total")
    for word in ${head_mem}; do
        word="$(capitalize "${word}")"
        head="${head}${word}\t"
    done
    colored_echo "${head}" "${bold}"
    echo -n "_____ "
    IFS=$'\n'
    lines=$(free -m | awk 'NR > 1')
    for line in ${lines}; do
        cur_line=$((echo "$line" | awk '{print $1, $2, $3, $4, $5, $6}'))
        avail=$(echo "$line" | awk '{print $7}')
        if [[ "$line" == *"Mem:"* ]]; then
            formatted_line "${cur_line}" "${orange}"
            echo -en "${orange}\t${avail}${reset}"
        else
            formatted_line "${cur_line}"
            echo -en "\t${avail}"
        fi
    done
}
```

We will first print the header of the table, then print each line (mem and swap) in a formatted way, and the 'Mem' will be in a special color like the following:

	Total	Used	Free	Shared	Buff/cache	Available
Mem:	3864	645	2894	3	324	2962
Swap:	1024	0	1024			

Finally, we will alert the user if there any the user utilized 80% of the memory or more, as the following:

```
| Your system's memory is 90 utilized, which may lead to:
|   - Instability.
|   - Perfomence Issues.
|   - Inability to launch new processes.
|
| Recommendations:
|   - Close unnecessary apps.
|   - Add more memory to your system.
```

- **Check Running Services:**

Which is listing the services that are currently running on the system, and this can be done using the ‘**systemctl**’ command, which is command that manage the services and check their status, we can use “**--type=service**” to list the services, so we will check the running services as follows:

```
services=()
length=0
while IFS= read -r service; do
    services+=("${service}")
    (( length++ ))
done << (systemctl --type=service | grep "running" | awk '{print $1}')
```

First, we will find the services that are currently running, then we will print these services and coloring the main services to distinguish them from the other services as the following:

```

for service in ${services[@]}; do
    color=${reset}
    if [[ ${service} == "system"* ]]; then
        color=${orange}
    fi
    echo -e "\t- ${color}${service}${reset}"
done

```

Here we are iterating over the array “services”, and if there are any services that starts with **system**, then we will color these services.

Finally, we will recommend the user to check the **running tasks (processes)**, or to see more details about the running services as the following:

```

| Recommendations:
|   - for more details, use 'systemctl --type=service'.
|   - use 'top' command to check the running tasks (processes).

```

- **Check recent system updates:**

Which is identify the last system updates and installations, and there are several ways to check them, one way is to open the ‘**~/.bash_history**’ file and retrieve the history that contains updates or installations from this file as the following:

```

updates=()
while IFS= read -r line; do
    updates+=("$line")
done < <(strings ~/.bash_history | grep -E 'apt update|apt install')

```

We will use ‘**strings**’ to extract the readable strings from this binary file, and then we will filter the output to give only the updates and installations, then we will simply print the last 10 commands from the ‘**updates**’ array (if there are 10 or more commands, otherwise, we will print the whole array) as the following:

```

end=0
if [ ${n} -ge 10 ]; then
    end=$((n - 10))
fi
for((i=n-1; i>=end; i--)); do
    line=${updates[$i]}
    echo -en "\t- sudo apt "
    if [[ ${line} == *"install"* ]]; then
        echo -n "install"
        install=$(echo "${line}" | awk '{print $4}')
        colored_echo " ${install}" "${orange}"
    else
        colored_echo "update" "${green}${bold}"
    fi
done

```

Finally, we will print some recommendations to the user like the following:

```

| Recommendations and Actions:
|   - To update the package list, use 'sudo apt update' command.
|   - When updating, consider to reboot, use 'sudo reboot' command.
|   - To list all the available packages, use 'apt list' command.

```

Performance Analysis

1. **Backup Script:** This script uses the **Zip** compression, which uses an efficient and fast compression algorithm, also, each task is **handled in a specific function** which makes the code easier to read, also provide a **user interaction** to give the **location** of the backup and decide whether to **compress** the backup or not, makes it more flexible, also the script includes **error handling** with validations to ensure an almost free errors while creating the backup, also this script update the information about each backup creation in **the log file**, which is useful to track the creation process.

2. **System Health Check Script:** This script provides also error handling but with recommendations to the user with a detailed and user-friendly report, also the commands '**df**', '**free**' and '**systemctl**' that we've used them in '**Disk Space**', '**Memory Usage**' and '**Running Services**' are optimized for quick retrieval and minimal impact on the system performance.

Scripts execution with \$PATH

I've included both scripts like a system built-in script, so that we can run the script just by typing the name of the script like the built-in system scripts ('**backup**' command: will execute the backup script, '**health**' command: will execute the system health check script), and this can be done by including the path of the scripts (which is /usr/scripts/bin) in the **\$PATH** system variable, this is one way to do that:

- Open the .bashrc file (or .bash_profile):

```
ahmdjaber@DESKTOP-MNB04TB:~$ vim ~/.bashrc
```

- Add this line to the end of the .bashrc file:

```
if [ -f /usr/share/bash-completion/bash_completion
. /usr/share/bash-completion/bash_completion
elif [ -f /etc/bash_completion ]; then
. /etc/bash_completion
fi
fi
export PATH="${PATH}:/usr/scripts/bin"
```

- And now, your directory must be included in the **\$PATH** variable:

```
ahmdjaber@DESKTOP-MNB04TB:~$ echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/scripts/bin
```

After adding the path to the scripts in **\$PATH**, the system will automatically investigate this directory when typing the script name without specifying the full path to the script.

Also, we can remove “**.sh**” from the scripts, because the system will automatically interpret the script as a shell script based on the ‘**#!**’ that is located at the beginning of the script.

Thanks

Ahmad Nabeel Al-Jaber